FindIt SDSU

Phase 1 — Technical Implementation Report

Alexander Hills · Andy Thai · Samueel Shadi

October 18, 2026 | Database Theory and Implementation

Live prototype: https://finditsdsu.pythonanywhere.com/

Application source: https://github.com/DellyCodez/FindIt-SDSU/tree/work/unified-site

Independent QA: https://github.com/DellyCodez/FindIt-SDSU/tree/test/database-tests

1. Project overview

At SDSU, losing something usually means checking group chats, asking friends, or hoping someone posts about it. The problem is that a lost-item post and a found-item post can be in completely different places. We wanted to make that process a little easier by keeping reports together in one searchable site.

FindIt SDSU lets someone post a lost or found item with a description, location, date, and photos. Other visitors can search the reports and see possible matches. If they recognize a found item, they can submit a claim explaining why they think it belongs to them. That explanation stays private until an administrator reviews it.

For Phase 1, we concentrated on making the main steps actually work with a database. One part we paid particular attention to was the claim process. More than one person can claim the same item, but the database must never mark two claims as approved. We built that rule into SQLite so it does not depend on someone using the website correctly.

2. How we built the application

The version we are presenting is on the work/unified-site GitHub branch and is hosted on PythonAnywhere. We used Python 3.12 and Flask for the server, Jinja for page templates, SQLite for the records, Pillow for images, and regular CSS and JavaScript for the interface. We kept the stack fairly small so we could focus on the database and the features that depend on it.

Where the main code lives

The public pages include Browse Items, report forms, item details, Possible Matches, a claim form, and the project information pages. Searching and filtering show data from SQLite, including reports added through the form; they are not just hard-coded example cards.

Most of the web routes are in app/__init__.py. That file handles form inputs, sessions, CSRF checks, and the routes used by the admin. Visitors do not need accounts to post or claim an item. Reviewing claims and hiding reports are limited to the password-protected admin pages.

We put the database operations in app/storage.py instead of writing separate SQL inside each page. The SQLiteStore class handles connections, parameterized queries, and updates such as creating reports or approving claims. This made it easier to test the database behavior separately from the interface.

The table definitions are in database/schema.sql. We also kept migrations so earlier versions of the project database can be upgraded to schema version 8 without starting over and deleting existing reports. On PythonAnywhere, the database file is kept outside the public repository.

Uploaded images are handled by app/photo_uploads.py. It checks each image, converts it to WebP, and saves the file outside the database. SQLite only stores the file name and other photo details needed to display it.

3. Database tables and relationships

Our final database has seven tables: users, items, categories, locations, item_photos, claims, and submission_attempts. Figure 1 is based on the ER diagram Andy prepared. We kept his layout and made the small Claims primary-key correction so the diagram uses claims.id, the same name as the real schema.

What the tables are used for

users keeps the names and email addresses associated with reports and claims. Each record has an integer id, and email addresses must be unique. These entries identify who submitted information; they are not student login accounts.

items is where the lost and found reports live. It connects to a reporter, category, and location through foreign keys. It also stores the title, description, date, Lost/Found type, Open/Returned status, whether the report is visible, and optional details such as brand, color, model, and a private contact email.

categories and locations hold the names used when someone creates a report. A report stores the corresponding ID instead of copying the category and location text into every row.

item_photos holds the metadata for uploaded pictures: the item it belongs to, file name, position in the gallery, MIME type, dimensions, and file size. The (item_id, position) combination must be unique, and each item can have at most three photos.

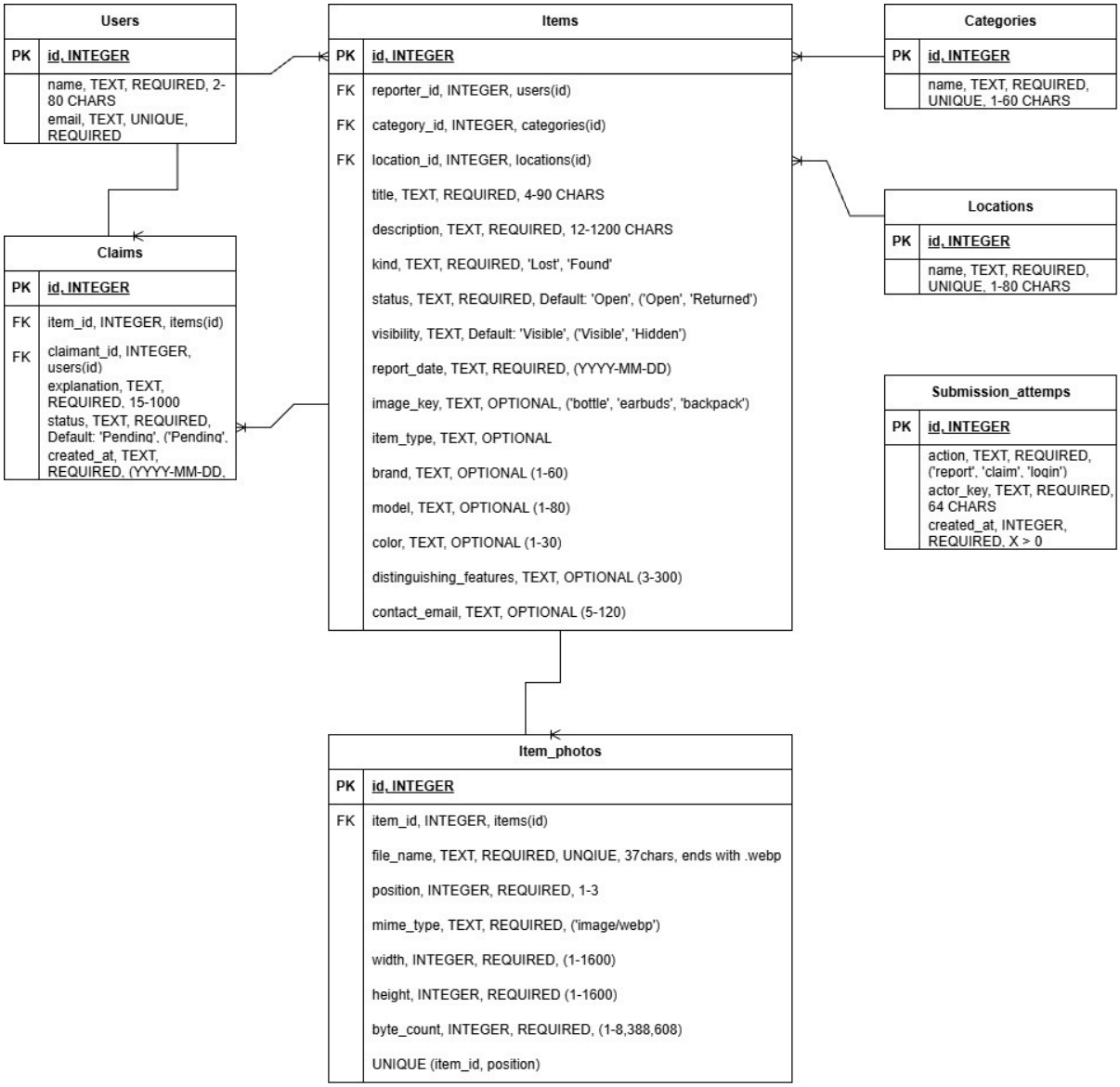
claims links a found item to the person claiming it. Along with the private explanation, it stores whether the claim is Pending, Approved, or Rejected. The database allows several claims on the same item but prevents more than one approval.

submission_attempts is used for basic rate limiting. It records the action, a keyed identifier, and the time of an attempt. We do not store the visitor's raw IP address in this table, and it does not need a foreign key to an item.

Why we separated the tables

These connections are mostly one-to-many. One person can report several items; each item has one category and location; and one found item can receive several claims. Photos are stored separately because a report might have none, one, or three. Deleting an item also deletes its photo metadata, while references to users, categories, and locations are protected.

This is where normalization helped us. Instead of typing “Library” into hundreds of item rows, the location is stored once and items reference it. Likewise, adding photos or new claims does not require extra columns in the items table. It keeps the records easier to maintain and avoids unnecessary repeated information.



4. Making sure claim data stays correct

We validate information in the forms, but we also check it in SQLite. Titles, descriptions, and other required fields have NOT NULL or length checks. An item is either Lost or Found, and a Found item may change from Open to Returned. Claims have three possible statuses: Pending, Approved, and Rejected. Foreign-key checks are enabled on each database connection.

The main database rule is the unique index below. It applies only to Approved claims, which means an item can have multiple pending claims but not two approved ones:

```
CREATE UNIQUE INDEX one_approved_claim_per_item
ON claims(item_id) WHERE status = 'Approved';
```

New claims must start Pending and point to an Open Found item. For an approval, app/storage.py starts a BEGIN IMMEDIATE transaction, checks that the claim is still pending, and changes its status. Triggers then mark the item Returned and reject the other pending claims for that item. Other triggers stop an approved item from being reopened and prevent claim identities from being changed afterward.

```
BEGIN IMMEDIATE;
```

```
UPDATE claims SET status = 'Approved' WHERE id = ?;
-- Triggers return the item and reject competing Pending claims.
COMMIT;
```

We used all three parts here—the transaction, the trigger, and the unique index—because they protect against different mistakes. The transaction keeps related updates together; the trigger handles the follow-up changes; and the index is a final check against a second approval.

5. SQL examples from the project

We used SQL for the actions people see on the site, and we also wrote a separate evaluation script to check queries. These are shortened examples from the repository. They show how the database concepts from class connect to actual FindIt features.

Showing category and location names

The browse page needs to show a report's category and location, not just their ID numbers. This query joins items to the two reference tables and returns recent found Drinkware reports:

```
SELECT i.id, i.title, c.name AS category, l.name AS location
FROM items AS i
JOIN categories AS c ON c.id = i.category_id
JOIN locations AS l ON l.id = i.location_id
WHERE i.kind = 'Found' AND c.name = 'Drinkware'
ORDER BY i.report_date DESC LIMIT 5;
```

Counting reports by location

For testing, we also used GROUP BY with COUNT and SUM to see how many reports belonged to each campus location and how many of those were Found:

```
SELECT l.name, COUNT(*) AS reports,
  SUM(CASE WHEN i.kind = 'Found' THEN 1 ELSE 0 END) AS found_reports
FROM items AS i JOIN locations AS l ON l.id = i.location_id
GROUP BY l.id ORDER BY reports DESC LIMIT 5;
```

Other database operations

Creating a report uses INSERT. The application looks up the selected reporter, category, and location IDs with subqueries, then inserts the new item. If pictures are attached, their metadata is inserted as part of the same operation. Claim review uses UPDATE, with triggers taking care of the related status changes.

We use DELETE for housekeeping in the rate-limit table. Older submission-attempt entries are cleared before the current request count is checked. There is no general public button that deletes reports:

```
DELETE FROM submission_attempts WHERE created_at < ?;
SELECT COUNT(*) FROM submission_attempts
WHERE action = ? AND actor_key = ? AND created_at > ?;
```

Index used for possible matches

The matching feature first narrows the database search to the opposite Lost/Found type, Open and Visible reports, the same category, and an appropriate date range. That limits how many records Python has to compare. We added the composite index idx_items_match_candidates(kind, status, category_id, report_date) to support this search and used EXPLAIN QUERY PLAN to see what SQLite actually did with it.

6. How the matching feature works

When someone reports a lost item, the application looks for found reports in the same category and within the allowed date window. It does the opposite for a found report. SQLite returns up to 300 recent candidates for Python to check, rather than comparing every item in the database.

Python then compares details that might help identify the same object: item type, location, date, shared descriptive words, brand, model, and color. It also rejects known conflicts. For example, a black phone and black earbuds should not match just because both are listed under Electronics. Common words such as “phone” are not enough by themselves.

Each candidate gets a similarity score, and only scores of at least 55 can appear. The page shows no more than three suggestions and explains the matching details. A suggestion is just a lead to look into, not proof that the items belong to the same person. Private contact information and claim explanations are never used for matching.

7. Photos and information privacy

People can attach up to three JPEG, PNG, or WebP photos to a report. The upload code checks the actual file contents, allows up to 5 MB per input file, and limits image processing to 20 million pixels. It fixes the image orientation if needed, resizes the longest side to at most 1,600 pixels, and saves a WebP version with a random filename.

We deliberately keep contact emails and claim explanations out of public listing queries. Only the protected admin views can retrieve the private claim details. Admins can also hide reports without deleting them. Hidden reports stop appearing in public searches, match suggestions, and photo requests.

Because posting is open to visitors, we added form validation, CSRF tokens, a hidden honeypot field, and SQLite-backed rate limits. Those controls help with a class prototype, but they do not solve every problem with a public service. Reports still appear immediately, so an actual campus rollout would need stronger moderation and privacy procedures.

8. Tests and database results

We tested the main application during development, including reporting, photo uploads, matches, claims, and database migrations. Andy separately added tests/test_database.py on the test/database-tests branch. His tests focus on the rules inside SQLite, and the GitHub Actions workflow ran the full 30-test suite successfully.

The database tests intentionally try invalid inputs and state changes: incorrect emails or dates, missing fields, foreign-key violations, changing a claim after submission, and approving competing claims. They also check what happens to photo records when an item is deleted. These cases matter because a form can appear to work while the underlying database still accepts bad data.

Results from the sample-data evaluation

The evaluation script makes a temporary database, adds 4,800 fictional background reports and six known Lost/Found pairs, and runs its checks there. That makes 4,812 reports in total. It found no foreign-key violations, found all six known pairs in the top three suggestions, and correctly excluded the phone-versus-earbuds example.

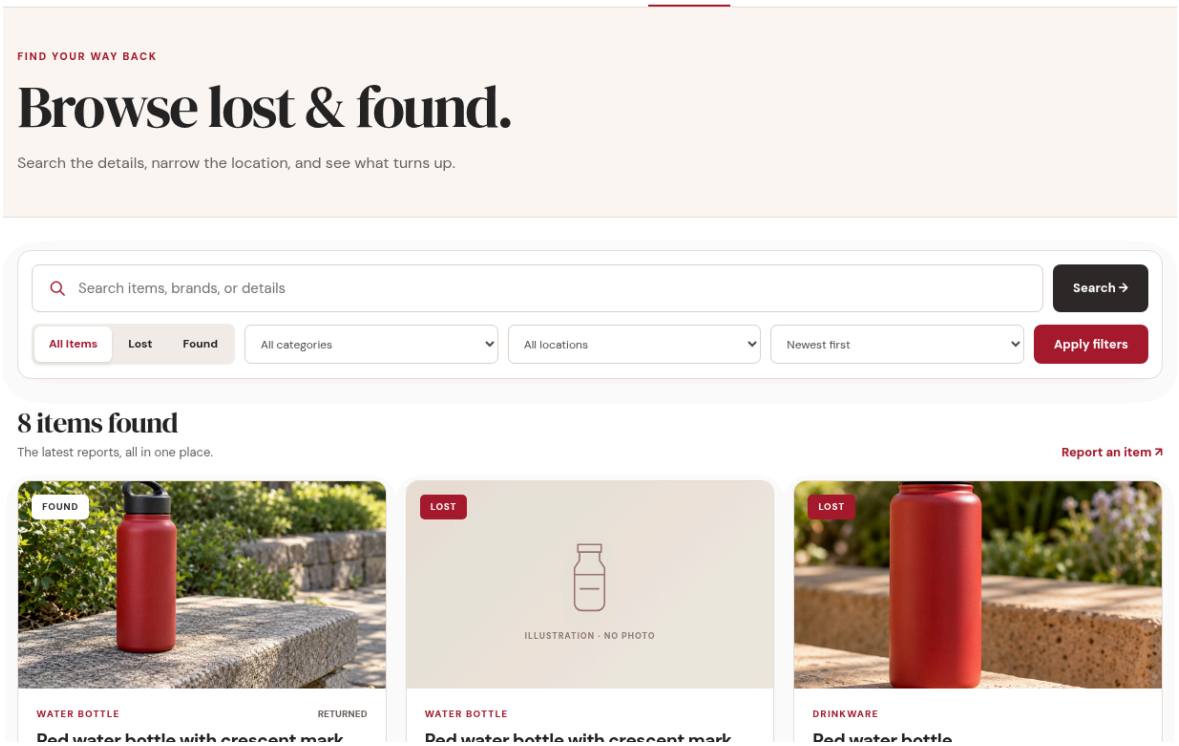
For the candidate search, the indexed query had a median time of 0.051 ms across 200 warm-cache runs. The same query with a forced full scan had a median time of 0.476 ms. The query plan showed index usage in one case and a scan with temporary sorting in the other. These measurements are useful for comparing those two SQL queries, but they do not tell us how fast the whole website will be for real users.

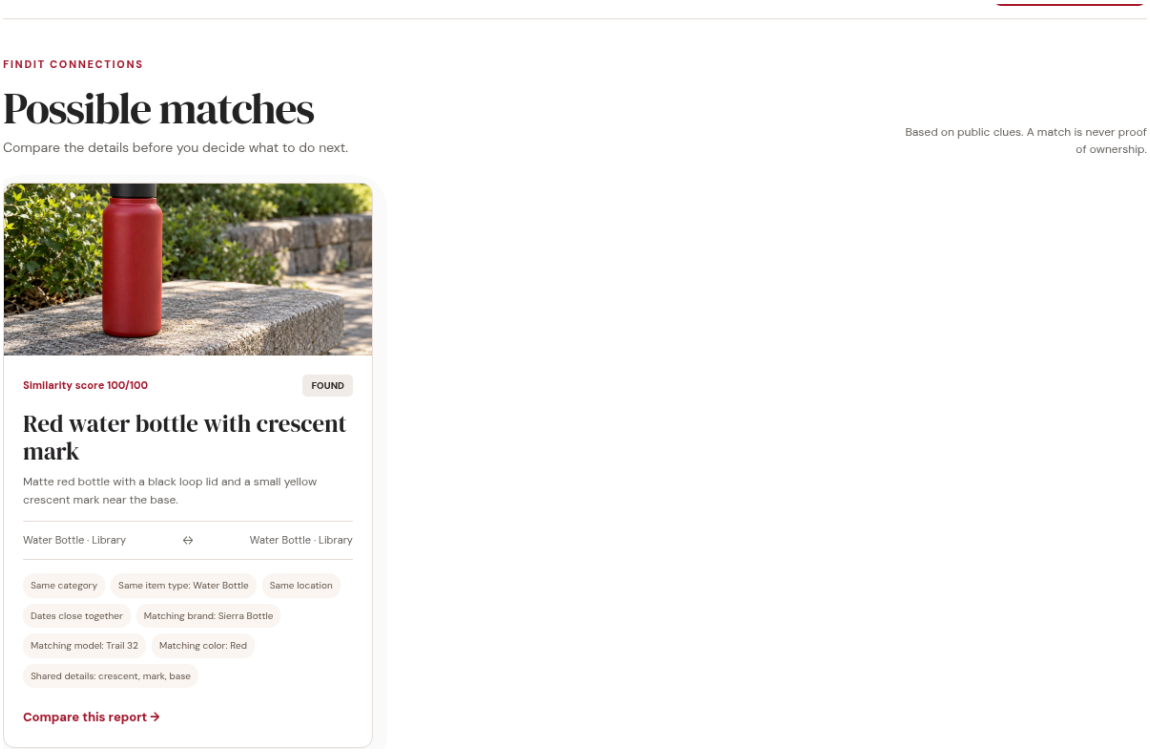
Independent QA proof: https://github.com/DellyCodez/FindIt-SDSU/actions/runs/39183211792

SQL evaluation source:
https://github.com/DellyCodez/FindIt-SDSU/blob/work/unified-site/database/evaluate.py

9. Screenshots and using the site

The screenshots show the actual unified application: Browse Items with its filters, a possible-match result, and the mobile reporting form. Figure 1 shows Andy's ER diagram. We included the screens here so someone reading the report can connect the database behavior to what the visitor sees.





Using FindIt as a visitor

1. Go to the live website and open Browse Items. Search for a lost or found report, or use the category and location filters. Click a report to see its description and available photos.

2. Select Report an Item, choose Lost or Found, and fill in the item's title, category, location, date, and description. Add photos if you have them, then submit the report.

3. Check the suggested Possible Matches. Compare the details yourself; the suggestions do not confirm ownership.

4. For a Found item that might be yours, open its page and submit a claim. Include enough private detail for an administrator to review it.

Admin demo

5. Open /admin/login and use the configured admin password. In claim review, approve a fictional test claim and show that the item becomes Returned while its competing pending claims become Rejected. Keep the password private during the presentation.

6. If needed, open moderation and hide or restore a fictional test report. You can also reload the page to show that the saved records remain in SQLite.

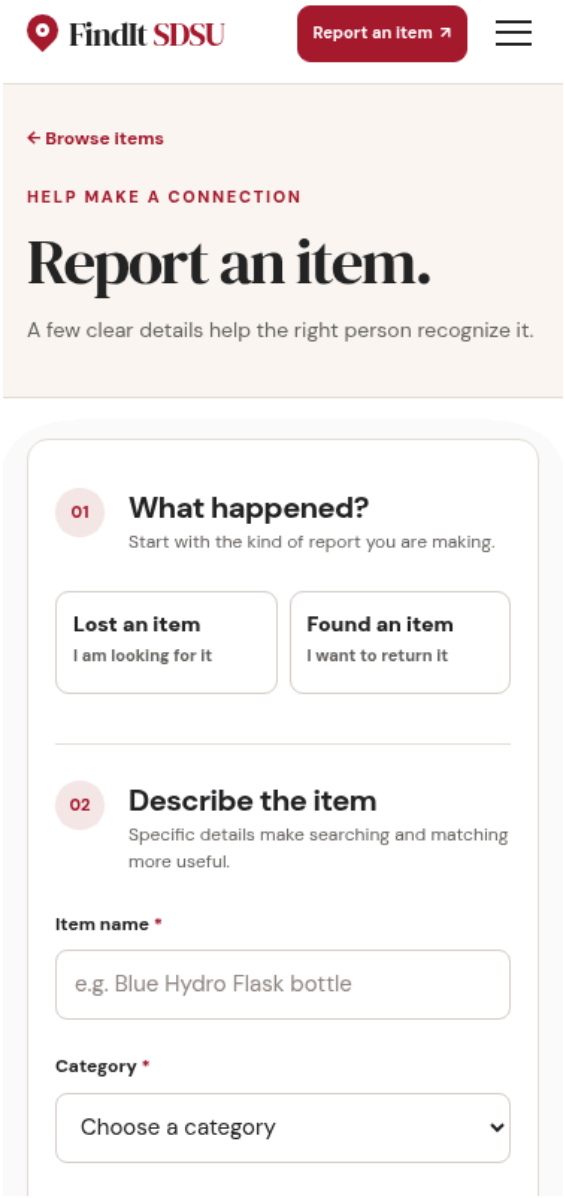


Figure 4. Mobile report form, showing the Lost/Found choice and required report details.

10. Running the application locally

The live version is at https://finditsdsu.pythonanywhere.com/. To run the same source locally, check out the work/unified-site branch. The commands below are for Python 3.12 on macOS or Linux:

```
git clone https://github.com/DellyCodez/FindIt-SDSU.git
cd FindIt-SDSU
git switch --track origin/work/unified-site
python3.12 -m venv .venv
.venv/bin/python -m pip install -r requirements-dev.txt
.venv/bin/python -m app.init_db
.venv/bin/python run.py
```

Open http://127.0.0.1:5000/ in your browser. Unless FINDIT_DB_PATH is set, the app keeps its local SQLite file in instance/. Re-running initialization applies supported migrations without deleting valid reports. On

PythonAnywhere, the database, SECRET_KEY, and PRESENTER_PASSWORD_HASH are kept outside the repository.

```
.venv/bin/python -m pytest -q
.venv/bin/python -m database.evaluate
```

The test and evaluation commands above use the development environment. The evaluation creates a separate temporary database, so it will not modify regular demo records. Image files live outside SQLite and must be backed up with the database. Never commit the administrator password or production data.

11. What is still missing

This is a working Phase 1 prototype, not a full campus service. It does not verify that someone is an SDSU student or that a claimant really owns an item. Visitors can post without accounts, and reports are public before an administrator reviews them. Matching also has limits: an item may be missed if it is filed under the wrong category, described too vaguely, or outside the date window.

If we continued developing FindIt, the next improvements would be account verification, email notifications, review of reports before publication, better long-term photo storage, and testing the matching rules with a larger and more realistic dataset. Those features are not part of what we are demonstrating for Phase 1.

12. Who worked on what

Alexander Hills handled the main Flask application and SQLite integration, brought the application and project pages together, and deployed the website to PythonAnywhere. His branch contains the reporting, browsing, matching, photo, claim, and moderation code, along with tests and migrations.

Andy Thai prepared the ER diagram and wrote independent database tests on a separate branch. His GitHub Actions run passed 30 tests.

Samueel Shadi prepared the original technical documentation and presentation materials that we used as the starting point for this report.

Sources and project links

Live application — https://finditsdsu.pythonanywhere.com/

Deployed source — https://github.com/DellyCodez/FindIt-SDSU/tree/work/unified-site

Authoritative schema — https://github.com/DellyCodez/FindIt-SDSU/blob/work/unified-site/database/schema.sql

Storage and transactions — https://github.com/DellyCodez/FindIt-SDSU/blob/work/unified-site/app/storage.py

Matching implementation — https://github.com/DellyCodez/FindIt-SDSU/blob/work/unified-site/app/matching.py

Database evaluation and query plans — https://github.com/DellyCodez/FindIt-SDSU/blob/work/unified-site/docs/database-evaluation.md

Andy's independent tests —
https://github.com/DellyCodez/FindIt-SDSU/blob/test/database-tests/tests/test_database.py

Successful 30-test CI run —
https://github.com/DellyCodez/FindIt-SDSU/actions/runs/39183211792

Contribution record —
https://github.com/DellyCodez/FindIt-SDSU/blob/work/unified-site/docs/alexander-contributions.md